Red text indicates my work

Spring 2026 MSML606 HW1 (Deadline 2/11, Wednesday)

Part I:

1) **Complexity Analysis**
   a) **$O(n^2)$**

```
for (i = 0; i < n; i++)                n iterations, O(n)
        for (j = 0; j < n, j += 3)     ⅓ n iterations → O(n)
                k += 1;                O(1) operation
```

   b) **O(log(n))**

```
cnt = 0                                O(1) operation
for (i = 1; i < n; i *=  2)            log₂n iterations → O(log(n))
        if i % 2! = 0:                 O(1) operation
                cnt++;                 O(1) operation
```

   c) **O(n)**

```
if (n % 3 == 0) {                      O(1) operation
        printf("abc");                 O(1) operation
} else {
        for (i = 0; i < n; i++)        n iterations, O(n)
                printf("cba");         O(1) operation
}
```

2) **Order by growth rate from slowest to fastest. Indicate any functions that grow at the same time: n^n, n!, (n+1)!, $\sqrt{n}$, e^n, log(n), $\log_2(n)$, $n^{1.5}$, log(n!), nlog(n)**
   a) **log(n), $\log_2(n)$, $\sqrt{n}$, log(n!), nlog(n), $n^{1.5}$, e^n, n!, (n+1)!, n^n**
   b) **In terms of Big O, log(n) and $\log_2(n)$ have the same growth rate and nlog(n) and log(n!) have the same growth rate. I used desmos to graph the rates**
3) **Can an algorithm with O(1/n) time complexity exist? What would it imply about the algorithms behavior as the input size increases**
   a) **O(1/n) means as input size increases, computation time decreases. As n reaches infinity, the time complexity should reach 0 and if the input size was 1 then time complexity would be 1. Because of this, I do not think an algorithm with O(1/n) time complexity exists. A single operation takes at minimum O(1) so there cannot be a time where an operation takes less than a single operation. If there was an algorithm that had O(1/n) running somewhere in it, it would be averaged into O(1).**
4) **Discuss whether traditional time and space complexity measures are still meaningful for evaluating modern machine learning algorithms, particularly neural networks**
   a) **In general, I believe traditional time and space complexity measures are still meaningful but not as meaningful as they were before. It provides an idea of the time but more complex hardware and optimization can adjust how things**

Red text indicates my work

are measured. Space complexity would probably matter more as if the measure is more than the space allowed then the algorithm won't run. For instance, running a program on the typical CPU from google collab runs much slower than running it on a T4 GPU. Neural networks takes all of this into consideration. Running a neural network on the phone has a hard limit on space for the model to run rather than running on a cloud. Also, neural networks can be parallelizable so even if an neural network has a big O complexity, the multiple tasks run at the same time would be more important.

Part II: Design your own sorting algorithm

1) **Clearly describe. Be explicit**
   a) **The idea of this sorting algorithm is to find the smallest number and the largest number in each pass and put them where they should be and then going inwards we repeat it until we get to the middle. The algorithm will only run if there are no duplicates in the input list. Each element must be unique.**
   b) **To explain this to a child: We spread out the numbered cards on the floor so we can see the numbers. We will build a bridge from the sides of the table going inwards. Find the lowest number and the biggest number. Put the smallest number (1) on the far left side of a table and put the largest numbered card (10) on the far right side of the table. Go back to the pile of cards on the floor. Now pick the smallest number left in the pile and the largest number left in the pile. Place the smallest number card next to the (1) and the largest number card next to the (10) so it fills the space between the cards. Repeat the process of finding the new smallest and new largest card from the pile and putting them on the insides of the bridge.**
2) **Psuedocode**

```
func (input list of size n):
    Sorted_list = empty list the same size as the input list of None
    Left = 0
    Right = n-1
    While left < right
        Sorted_list[left] = min(input list)
        Sorted_list[right] = max(input list)
        Input list = input list without the min and max values
        Left += 1
        Right -= 1
        If there is only one element left in the input list
            Assign that value as the left value of sorted list
    Return sorted_list
```

Red text indicates my work

3) **Step by step trace using example list of 10 elements**

```python
unsorted_list = [9, 1, 4, 3, 6, 8, 2, 5, 10, 7]

def sorting_alg(input_list):
  print(f"Input list: {input_list}\n")
  input_set = set(input_list)
  sorted_list = [None]*len(input_set)
  left, right  = 0, (len(input_list) - 1)
  while left < right:
    min_value, max_value = min(input_set), max(input_set)
    print(f"Min value: {min_value}, Max value: {max_value}")
    sorted_list[left] = min_value
    sorted_list[right] = max_value
    input_set.remove(min_value)
    input_set.remove(max_value)
    left += 1
    right -= 1
    print(f"Adjusted input set: {input_set}")
    print(f"Updated sorted list: {sorted_list}\n")
    if len(input_set) == 1:
      sorted_list[left] = input_set.pop()
  return sorted_list

print(f"Sorted list: {sorting_alg(unsorted_list)}")
```

```
Input list: [9, 1, 4, 3, 6, 8, 2, 5, 10, 7]

Min value: 1, Max value: 10
Adjusted input set: {2, 3, 4, 5, 6, 7, 8, 9}
Updated sorted list: [1, None, None, None, None, None, None, None, None, 10]

Min value: 2, Max value: 9
Adjusted input set: {3, 4, 5, 6, 7, 8}
Updated sorted list: [1, 2, None, None, None, None, None, None, 9, 10]

Min value: 3, Max value: 8
Adjusted input set: {4, 5, 6, 7}
Updated sorted list: [1, 2, 3, None, None, None, None, 8, 9, 10]

Min value: 4, Max value: 7
Adjusted input set: {5, 6}
Updated sorted list: [1, 2, 3, 4, None, None, 7, 8, 9, 10]

Min value: 5, Max value: 6
Adjusted input set: set()
Updated sorted list: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Sorted list: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

4) **Describe the abstract data type and the specific operations you are using (list/stack/queue). List the specific operations of that ADT that your algorithm relies on. Briefly explain why this ADT is appropriate for your logic**

   a) **For this algorithm I am using a list and a set. Since I know the input list will not contain any duplicates, I can use set to cut down on runtime for specific operations. I used the operation set() to construct my set, min/max to find the minimum and maximum, remove() to remove specific elements and len(). This adt is appropriate because I know the algorithm only works with unique values for the input and using a set makes it easier and faster to remove the elements. For the list i am using indexing in order to put the elements in the correct order in the sorted list**

Red text indicates my work

5) **Perform formal big O analysis of your algorithm**

```
def sorting_alg(input_list):
    input_set = set(input_list)
    sorted_list = [None]*len(input_set)
    left, right  = 0, (len(input_list) - 1)
    while left < right:
        min_value, max_value = min(input_set), max(input_set)
        sorted_list[left] = min_value
        sorted_list[right] = max_value
        input_set.remove(min_value)
        input_set.remove(max_value)
        left += 1
        right -= 1
        if len(input_set) == 1:
            sorted_list[left] = input_set.pop()
    return sorted_list
```

Worst, Best, Average Case: $O(n^2)$

In every case, the algorithm runs the while loop n/2 times which is O(n) complexity. However, running the min and max function on the set are basically O(n) for loops. This means that the total time complexity is $O(n^2)$. Since we use a set on the input list, it does not matter if it got imputed sorted or not, the algorithm will still run. Making the input list a set helped with preventing the remove operation from being O(n) but the algorithm is still $O(n^2)$.

Space complexity: O(n)

The algorithm creates a set of the same size as the input list and a list for the sorted values. It does not require any other storage for things other than O(1) things such as the min/max value and the right/left values. The algorithm is not in-place as we create a whole new list to store the sorted list.